5.0 Funciones y Modulos

Partiendo en pedasos nuestros programas :D

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Rio Negro

Repaso express de la clase anterior

- ¿Cual es la diferencia entre intérprete e ide?
- ¿Cómo ejecutamos nuestros scripts python?
- ¿Que es un bucle while?
  - ¿Cual es la diferencia con un for?
  - ¿Como detengo un bucle for?
- ¿Que es una lista?
  - ¿Puedo agregar elementos, Modificarlos o eliminarlos?

METODOS
O
FUNCIONES

¿Que es una función?

Las funciones son una pieza clave en los lenguajes de programación, todos los lenguajes modernos las implementan de alguna u otra manera.

Nos permite agrupar codigo para construir **bloques reutilizables** que encapsulan una tarea específica, puede recibir datos de entrada, procesarlos y devolver un resultado si es necesario.

De esta forma podemos crear software más modular, entendible y mantenible.

Sorpresa: las estamos usando cuando escribimos **print**, **input** o **type**.

Funciones: declaración simple

1. La definición de una función comienza siempre con la palabra clave “def”.
2. A continuación el nombre que le queremos poner a la función
3. Siempre terminamos con paréntesis “()” y dos puntos “:”.

4. Las funciones puedes encapsular lógica más compleja

1 def: indicador de declaración de función

nombre de función 2

```
def saludar():
    print("Hola")
saludar()
```

codigo que se ejecuta cuando se llama a la función 3

4 Llamado a función

```
def bienvenida():
    nombre = input("Ingrese su nombre: ")
    print(f"Hola {nombre} 😃")
    print("Bienvenido a mi programa")

bienvenida()
```

Funciones con argumentos

Los argumentos nos permiten pasarle información a una función.

Los argumentos se definen dentro de los paréntesis de definición de la función

Los argumentos pueden ser de cualquier tipo

Dentro la función podemos utilizar los argumentos como si fueran variables.

Podes pasar tantos argumentos como necesites.

Argumento

```
def saludar(nombre):
    print(f"Hola {nombre} 😃")

saludar("Juan")
```

```
def saludar(nombre_1, nombre_2):
    print(f"Hola {nombre_1} y {nombre_2} 😃")

saludar("Juan", "Pedro")
```

Funciones con argumentos por defecto

Los argumentos son obligatorios siempre y cuando no tengan un valor por defecto.

El valor por defecto será utilizado por la función en caso de no definirlo al llamar una función

```
def saludar(nombre = "Juan"):
    print(f"Hola {nombre} 😃")

saludar()
saludar("Pedro")
```

```
def saludar2(nombre_1 = "Juan", nombre_2 = "Pedro"):
    print(f"Hola {nombre_1} y {nombre_2} 😃")
saludar2()
saludar2(nombre_1="Pancho")
saludar2(nombre_2="Martin")
```

```
def sumar(a = 0, b = 0):
    print(f"La suma es: {a} + {b} = {a + b}")

sumar(1, 2)
sumar(a=1)
sumar(b=2)
```

Funciones con retorno

El retorno de una función nos permite devolver resultados al llamar a una función.

Esto se hace la palabra clave “return” y el contenido a retornar.

```
def saludar(nombre):
    return f"Hola {nombre} 😃"

saludar("Juan")
print(saludar("Pedro"))
```

retorno

```
def sumar(a, b):
  return a + b
resultado = sumar(1, 2)
print(resultado)
```

```
def es_mayor(edad):
    if edad >= 18:
        return True
    else:
        return False

print(es_mayor(15))
print(es_mayor(20))
```

Funciones = Mejor código (si las usamos bien)

Las funciones nos permiten escribir código más descriptivo

```
def trae_documento():
    return input("Trae documento [si/no]: ") == "si"

def ingresar_edad():
    return int(input("Ingresar edad: "))

def puede_pasar(documento, edad):
    return documento == True and edad >= 18

documento = trae_documento()
edad = ingresar_edad()
if puede_pasar(documento, edad):
    print("Puede pasar")
else:
    print("No puede pasar")
```

Ejemplos de aplicación – ¿Que está pasando acá?

```python
def evaluar_energia(nivel = 0):
    if nivel > 7:
        print("Modo máquina activado 🚀")
    elif nivel > 4:
        print("Se puede laburar 👍")
    else:
        print("Necesita café urgente ☕")

evaluar_energia()
evaluar_energia(9)
evaluar_energia(6)
evaluar_energia(2)
```

```python
def acceso(clave):
    if clave == "1234":
        return "Acceso permitido"
    else:
        return "Acceso denegado"

acceso("1234")
```

```python
def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b

def multiplicar(a, b):
    return a * b

sumar(restar(1, 2), multiplicar(3, 4))
```

Ejercicio

1. Crear una función pedir_comida() que:
    - pida una comida al usuario
    - si el valor está vacío, vuelva a pedirlo
    - cuando sea válido, lo devuelva
2. Crear una función obtener_precio(comida) que:
    - reciba una comida
    - tenga el siguiente menú:
        - hamburguesa → 100
        - milanesa → 125
        - pizza → 140
    - imprima el nombre y precio
    - devuelva el precio
    - si no es válida, indicar error y devolver 0
3. Llama a las funciones:
    - pedir la comida
    - obtener el precio

MODULOS

MÓDULOS

Los módulos nos permiten separar nuestro programa en diferentes archivos.

La combinación de funciones con módulo son muy útiles para ordenar nuestros programas.

En los módulos podemos almacenar variables y funciones que luego podemos importar desde nuestro programa.

Para crear nuestro primer módulo hagamos los siguientes pasos:

1. Crear archivo funciones.py con el siguiente codigo:

```
def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b

def multiplicar(a, b):
    return a * b
```

2. En la misma carpeta crear archivo miprograma.py

```
import funciones # nombre del modulo sin .py

funciones.sumar(1, 2)
funciones.restar(1, 2)
funciones.multiplicar(1, 2)
```

3. Ejecutamos nuestro programa con **python.exe miprograma.py**

Mecanismos de importación

Importando modulo completo

```
import funciones # nombre del modulo sin .py

funciones.sumar(1, 2)
funciones.restar(1, 2)
funciones.multiplicar(1, 2)
```

Importación selectiva de metodos del modulo

```
from funciones import sumar, restar

sumar(1, 2)
restar(1, 2)
multiplicar(1, 2)
```

Importando todos los metodos del modulo

```
from funciones import *

sumar(1, 2)
restar(1, 2)
multiplicar(1, 2)
```

Ejercicio

1. Mover las funciones pedir_comida y obtener_precio(comida) al archivo utilidades.py
2. Importar las funciones desde pedidordecomida.py
3. Llamar a las funciones
   - Pedir la comida
   - Obtener el precio

PDFS y TAREAS

Todo el material en https://github.com/Ymil/unrn-programacion-1/

FIN

Volvemos el miércoles a las 17